

This is Topic, which is Vague to be fully discovered yet

Emergent behaviour

↳ Ability of a model to perform tasks that the model wasn't explicitly trained to perform;

STAGES of Building LLM

Step 1

Implement Data sampling understand basic mechanism

Data Preparation and Sampling

Attention Mechanism

LLM architecture

Building an LLM

Pretraining

Step 2

Pretrain an LLM on unlabeled data

Training loop

Foundation Model

Model Evaluation

Load pretrained weight

Finetune

Step 3

Most Young Developer

Direct deploy application used

Step 3

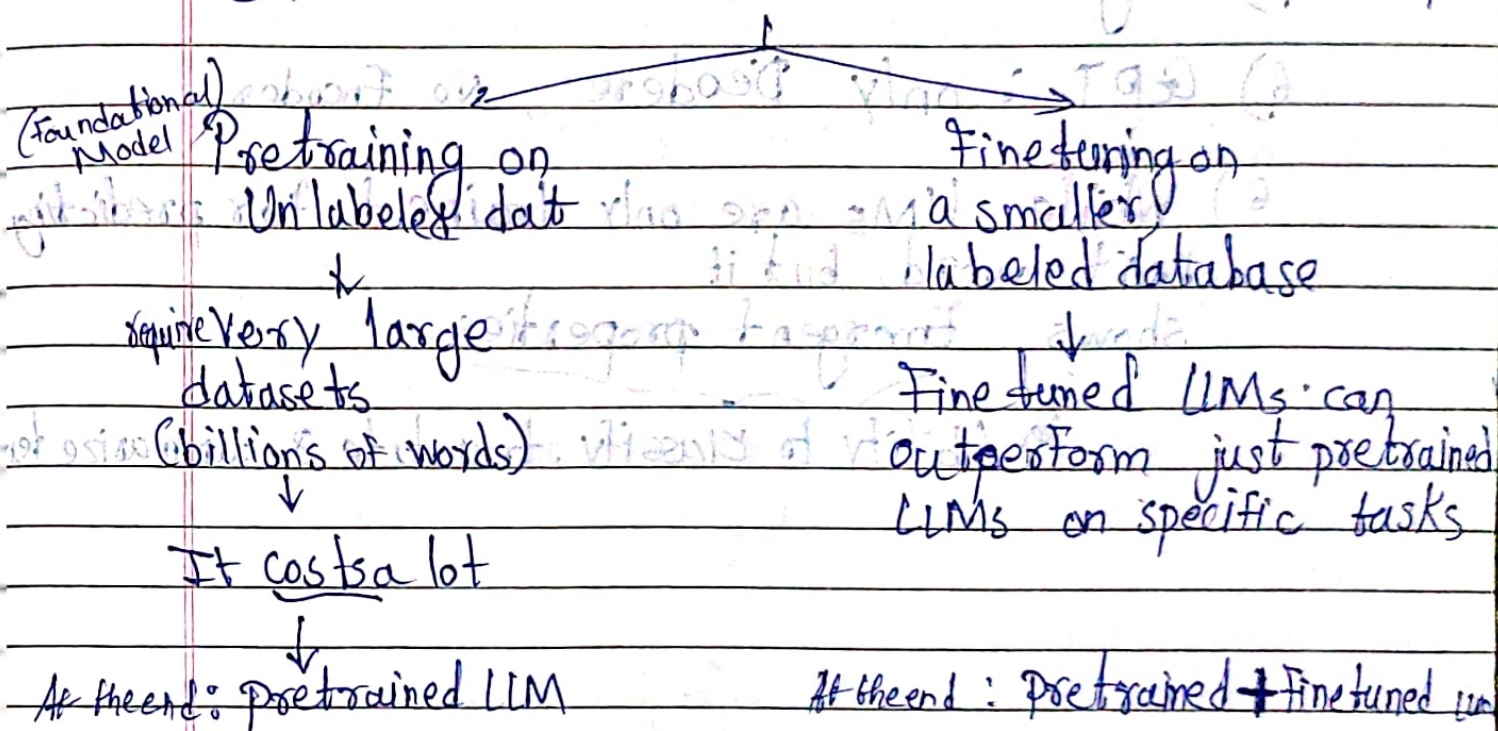
But Not confident how it Baked work

Classifier

Personal Assistant

1) LLM have transformed the field of NLP. They have led to advancement in generating, understanding & translating human language.

2) All Modern LLM's are trained in 2 Main steps



Finetuning is required to implement LLM for specific tasks

3) LLM are based on Transformer Architecture

4) Key idea: Attention Mechanism

↳ gives LLM selective access to whole input sequence when generating output one word at a time

Basically, it allows the LLM to understand the importance of the words and not just the word in the current sentence but in the previous sentences, which have come long before also. because, context is also important in predicting next word.

Attention mechanism allows the LM to give access to the entire context and selector give weightage to which words are important in predicting the next word.

5) Original Transformers: Encoder & Decoder

6) GPT: only Decoder, No Encoder

6) While LMs are only trained for predicting next word, but it shows Emergent properties

↳ Ability to Classify, Translate & summarize text

* Data Preparation & Sampling

How do you prepare input text for training LMS?
 → By tokens

* TOKENISATION

Step 1: a) Splitting text into individual words and subword

Step 2: b) Convert tokens into token IDs

Step 3: c) Encode token IDs into vector representations

Ex.

Input

Token embedding (step 3)

Token IDs (Step 2)

Tokenized Text (Step 1)

Input text

Input text: "This is an example"

Input text: "Hello world"

Input text: "I am a student"

Step 1: Download your text file/book (ex. The verticf)

load in python

```
with open("the-verdict.txt", "r", encoding="UTF-8") as f:
    raw_text = f.read()
```

Program

```
print("Total number of character:", len(raw_text))
print(raw_text[:99])
```

Output

Total number of character: 20479
I HAD always thought Jack Gisburn...

Our goal is to tokenize this 20,479 character short story into individual words & special character we can turn into embedding for LLM training

Splitting this text to obtain a list of tokens?

Import re

This library help to split string after certain repetition of particular character

Program

```
text = "Hello, world. This, is a test."
result = re.split(r'(\s)', text)
```

```
print(result)
```

Output

['Hello,', ' ', 'world.', ' ', 'This,', ' ', 'is,', ' ', '']

It means we are splitting for white space

So in program, when it scan print statement, left to right wherever it find white space it will split

Now

What we want to do

we want split on whitespace (), commas & period

Program `result = re.split(r'([.,])\s', text)`
`print(result)`

output `['Hello', ',', ' ', 'World', '.', ' ', 'This', ',', ' ', '']`

We can see that the words & punctuation characters are not separate list entries just as we wanted

But still whitespace character is present, we can remove the redundant character safely as follows

Program `result = [item for item in result if item.strip()]`
`print(result)`

output `['Hello', ',', 'World', '.', 'This', ',', 'Is', 'a', 'test', '.']`

No whitespaces in output

Removing whitespaces or Not (Not always)

↳ when you developing simple tokenizer,
In Removing whitespaces or encoding them depends on your requirement

1] Removing : 1] It reduce Memory & Computation requirement

2] Not Removing : 1] Keeping them can be useful if we train models that are sensitive to the exact structure of the text
(Ex. Python code, which is sensitive to indentation and spacing)

So Now what if there will be '?', '!', '(-)' etc symbol.

We have separate them also.

Program:

```

text = "Hello, world. Is this -- a test?"
result = re.split(r'([.,:;?_!'"()'\']|--)', text)
result = [item.strip() for item in result if item.strip()]
print(result)

```

Output: ['Hello', ',', ' ', 'world', '.', ' ', 'Is', ' ', 'this', ' ', '--', ' ', 'a', ' ', 'test', '?']

Now we got basic tokenizer working, let's apply it to Edith Wharton's entire short story. (The Verdict book's writer)

Program: `X = re.split(r'([.,:;?!"()'\s])', raw_text)`
`X = [item.strip() for item in preprocessed if item.strip() != '']`
`print(X[30])`

Output: `['I', 'HAD', 'always', 'thought', 'Jack', 'Leisburn', ...]`

Program: `print(len(X))`

Output: 4690

So Tokens are ready

Now,
Converting it to idf

So basically we tokenized that book and assigned it to a python variable called X

Step 2: tokens $\rightarrow$ Id's

Complete training Dataset $\rightarrow$ Tokenized Text $\rightarrow$ Vocabulary ^{usually}

"The quick brown fox jumps over the lazy dog."

The	quick	brown	$\rightarrow$	0
fox	jumps	dog	$\rightarrow$	1
over	the	fox	$\rightarrow$	2
lazy	dog	jumps	$\rightarrow$	3
over	the	fox	$\rightarrow$	4
lazy	dog	quick	$\rightarrow$	5

Each unique token is mapped to an unique integer called token ID

Let's now, Create a list of all unique tokens & sort them alphabetically to determine the vocabulary size.

all_words = sorted(set(X))

Program: Vocab_size = len(all_words)

print(Vocab_size)

Output: 1130

enumerate!
Command in python
it takes all words & assign
number

vocabulary only
consist of unique
words

$\therefore \text{len}(\text{token}) > \text{len}(\text{vocab})$

After determining that the vocabulary is 1130 via the above code,

We create the Vocabulary dictionary

$\text{Vocab} = \{ \text{token} : \text{integer} \}$ For integer, token in $\text{enumerate}(\text{all_words})$

Program { For i, item in enumerate(Vocab.items()):
print(item)
if i >= 50
break

Output:

('!', 0)

('"', 1)

('\'', 2)

('(', 3)

(')', 4)

(';', 5)

('.', 6)

(':', 7)

('A', 8)

('Among', 9)

('And', 10)

('As', 11)

('Be', 12)

Every Key in dictionary is Unique

So here,
The Vocabulary dictionary consist of individuals tokens assigned to ~~tokens~~ unique integer value. label as token id

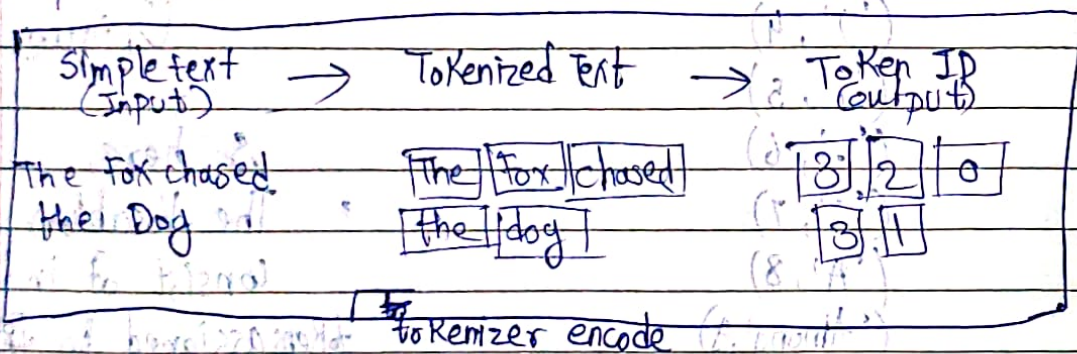
Later, w

When we want to convert output of an LLM from numbers back into text, we also need a way to turn token for this, we can create an inverse version of the vocabulary that maps token ID's back to corresponding text tokens.

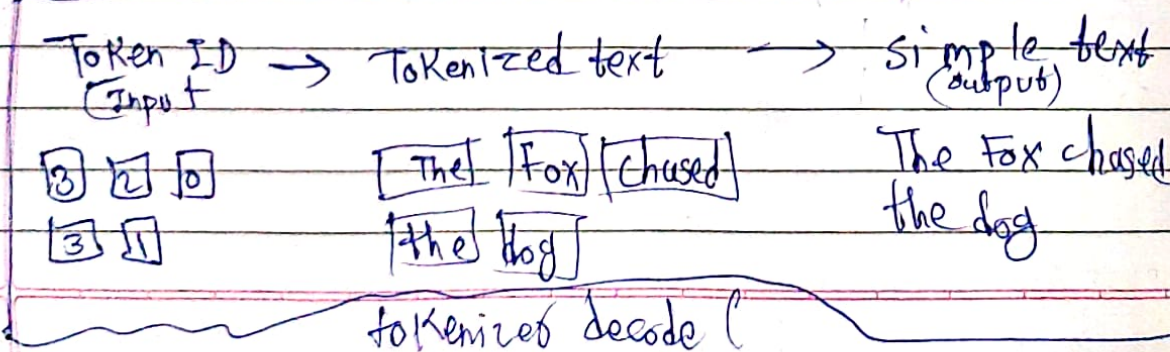
Let's Implement complete tokenizer class in Python

The class have,

1) Encode method: that splits text into tokens & carries out the string-to-integer mapping to produce



2) Decode Method: that carries out the reverse in Integer-to-string mapping to convert the token ID.



Preprocessed = X

PAGE NO.	
DATE	/ /

```
class SimpleTokenizerV1:
    def __init__(self, Vocab):
        self.str_to_int = Vocab
        self.int_to_str = {i:s for s,i in Vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([.,:;?!"'(){}[\]`~!@#$%^&*~])', text)
        preprocessed = [item.strip() for item in preprocessed
                        if item.strip()]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        # Replace spaces before the specified punctuation
        text = re.sub(r'([.,:;?!"'(){}[\]`~!@#$%^&*~])', r'\1', text)
        return text
```

```
tokenizer = SimpleTokenizerV1(Vocab)
```

```
text = " " "It's the last he painted, you know,"  
Mrs. Geisburn said with pardonable
```

```
ids = tokenizer.encode(text)
print(ids)
```

output: [1, 56, 7, 856, 988, 602, 533, 748, 5, 1126, 5, 1, ...]

This is encoder's work

Program output: tokenizer.decode(ids) # This is decoder's work

"It's the last he painted, you know," Mrs. Geisburn ...

So far,
we implemented a tokenizer capable of tokenizing & de-tokenizing text based on snippet from the training set;

But what if text given is not part of training set.

Program:

```
text = "Hello, do you like tea?"
print(tokenizer.encode(text))
```

'Hello' is not Present in training data

Output: Error

∴ The problem is that the word "Hello" was not used in the ~~verdict~~ short story.

Hence, it not contained in the Vocabulary.

This highlights the need to consider large & diverse training set to extend the vocabulary when working on LLMs

So if user used a 'word', which is not part of its dataset,

then it will show error,

that's why it is used large amount of data

So for Solving this,

Special Context token used.

* Adding Special Context Tokens.

In the previous section, we implemented a simple Tokenizer & applied it to a passage from the training set.

In this section, we will modify this tokenizer to handle Unknown words, from dataset.

In particular,

We modify the vocabulary & tokenizer we implemented in the previous section, SimpleTokenizerV2, to support two new tokens,

<UNK> & <end-of-text>

We can modify the tokenizer to use an <UNK> token if it encounters a word that is not part of the vocabulary.

Exe

Text

tokens

Existing Vocabulary

The Fox chased → [The] [Fox] [chased] → chased → 0
 the dog → [the] [dog] → dog → 1
 Fox → 2
 the → 3

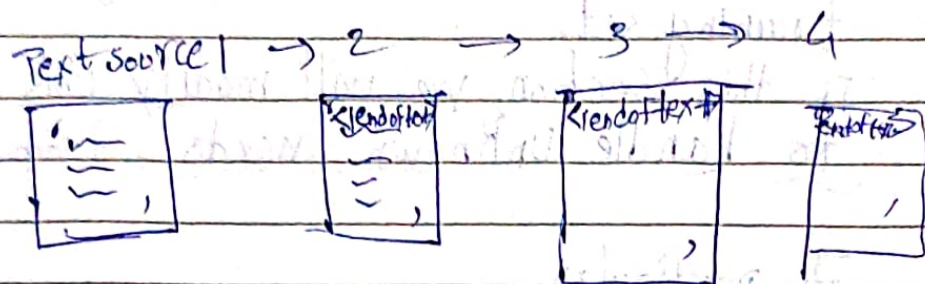
last 2 tokens } <UNK> → 783
 <end-of-text> → 784

So if there's any word, like "quickly" is added,

The dog chased the fox, quickly. } then quickly gets 783 (last token)
 quickly → 783

<endof text 1>

↳ when working with multiple text source, we add 1 <endof text> token between texts.



∴ These <endof text> token act as a marker, signaling the start of end of a particular segment.

∴ This leads to more effective processing & understanding by the LCM.

So

This is also used to relate unrelated ~~text~~ text.

Adding this tokens.

```
all_tokens = sorted(list(set(preprocessed)))
```

```
all_tokens.extend(["<endof text 1>", "<unk>"])
```

```
vocab = {token: integer for integer, token in enumerate(all_tokens)}
```

```
# Previous length of vocab : 1130
```

```
len(vocab.item())
```

```
# New vocab
```

```
∴ output : 1132
```


Other important tokens:

[BOS]: Beginning of Sequence (Represent start of text)

[EOS]: End of sequence (Position end of text)

[PAD]: Padding tokens: When LLM trained in batches, the batch sizes can be larger than one, the batch might contain text of varying length. To ensure all text in batch have same length & shorter text are extended (padded) using this token, up to the length of the longest text.

GPT model use only $\langle \text{end of text} \rangle$, to keep simplicity.

GPT model also not use $\langle \text{unk} \rangle$ token, for word out of vocabulary.

instead they use byte pair encoding tokenizer which break down words into subword units.

* Byte Pair Encoding (BPE)

First see,

Tokenization Algorithm

Word based

Sub-word based

Character based.

↓
Every word is Token.

↓
Every character is token

Eg

fox eat dog → [fox] [eat] [dog]

Eg

My hobby is → [M] [y] [h] [o] [b] [b] [y] [i] [s]

Problem: what about out of vocabulary words, or different meanings of similar word. or

[boy] [boys], these similarity is not captured

∴ Very small vocabulary, cause every language has ~256 ~~char~~ fixed characters

↳ No problem of out of vocab

Problem: The meaning of word completely lost. Also tokenized sequence is much longer than initial raw text.

• Sub-word based Tokenisation:

Rule 1: Do not split frequently used words into smaller subwords

Rule 2: split the rare words into smaller, meaningful subwords.

Eg.

~~by~~ boy : Should Not split

boys : Should be split into 'boy' & 's'

- The subword splitting helps model learn that different words with same Root Word as 'token' like 'tokens' and 'tokenizing' are similar in meaning.

- It also helps the model learn that 'tokenization' and 'modernization' are made up of different root words, but have the same suffix 'ization' & are used in same syntactic situation.

* BPE is subword tokenization algo.

* BPE Algorithm

↳ 1994

↳ data compression Algo

↳ It Most common pair of consecutive bytes of data is replaced with a byte that does not occur in data.

BPE

DATE	/ /
------	-----

Example:

Original data: aaabddaaabac

a) The byte pair 'ad' occurs the most.
we will replace it with 'z' as
'z' does not occur in the data

b) Compressed data: Zabdzabac

The next common byte pair is 'ab', we
will replace this by 'y'.

c) Compressed data: ZydzYac

Only byte pair left
occurs only once, so we
Not encode it.

Next 'zy' pair Compressing

d) Compressed data: WdWac

So

How BPE Algorithm used for NLPs?

BPE ensures that most common words in
the vocabulary are represented as a single token,
while rare words are broken down into two
or more subword tokens.

2) Example:

* Let us consider the below dataset of words,

{ "old": 7, "older": 3, "finest": 9, "lower": 4 }

* Preprocessing: We need to add end of token "</w>" at the end of each word.

{ "old </w>": 7, "older </w>": 3, "finest </w>": 9, "lower </w>": 4 }

These are Not Id, It represent No. of time word occurred in dataset

*

* let us now split the words into characters & count their frequency.

Number	Token	Freq.
1	</w>	23
2	o	14
3	i	14
4	d	10
5	e	16
6	r	3
7	f	9
8	n	9
9	s	9
10	t	13
11	w	13
12		14

* Next step is look for most the most frequent Pairing.

Merge them & perform the same iteration again & again until we reach the token limit or iteration limit.

Iteration 1:

start with second most common token that is 'e'. (cause first is <1w>)

∴ Most Common byte pair starting with 'e' ∴ 'es'.

Now

Number Tokens Frequency

1	<1w>	23
2	ok	14
3	l	14
4	d	10
5	e	16-13=3
6	r	3
7	F	9
8	i	9
9	n	9
10	es	13-13=0
11	st	13
12	1w	11
13	es	9+4=13

Now,

Iteration 2: Merge the tokens "es" and "t" as they have appeared 13 times in our dataset.

Number	Token	Frequency
1	< /w>	23
2	o	14
3	l	14
4	d	10
5	e	$16-13=3$
6	f	9
7	r	9
8	i	9
9	n	9
10	s	$13-13=0$
11	t	$13-13=0$
12	w	4
13	es	$9+4=13-13=0$
14	est	13

Iteration 3:

Now let us look at "< /w>" token, we see that "est < /w>" has appeared 13 times

Number	Token	Frequency
1	< /w>	$23-13=10$
2	o	14
3	l	14
4	d	10
5	e	$16-13=3$
6	f	9
7	r	9
8	i	9
9	n	9
10	s	$13-13=0$
11	w	$13-13=0$
12	es	4
13	est	$9+4=13-13=0$
14	est	$13-13=0$
15	est < /w>	13

Why we include $\langle w \rangle$ cause, if we not include,

it is hard to differentiate between the estimate & highest

Iteration 4:

'0' & '1' has appeared 10 times

Number	Token	Frequency
1	$\langle w \rangle$	$23-13=10$
2	0	$14-10=4$
3	1	$14-10=4$
4	d	10-10=0 10
5	e	$16-13=3$
6	f	3
7	g	9
8	i	9
9	h	9
10	es	$13-13$
11	st	$13-13$
12	t-w	4
13	es	$9+4=13-13=0$
14	est	$13-13=0$
15	test $\langle w \rangle$	13
16	ol	$7+3=10$

Next.

Iteration 5: '01' & '8' has appeared 10 times.

Number	Token	Frequency
1	<w>	10
2	o	4
3	i	4
4	d	3
5	e	3
6	r	3
7	f	9
8	i	9
9	n	9
10	s	9
11	t	9
12	w	4
13	es	4
14	est	9
15	est<w>	13
16	ol	0
17	old	10

Iteration 6:

"f", "i", "n" appears 9 times. But we just have one word with these characters. So, we are not merging them. (why would we use 'old', cause in coming in different form/words)

Let us remove tokens with zero count

Number	Token	Frequency
1	<w>	10
2	o	4
3	i	4
4	e	3
5	r	3
6	f	9
7	i	9
8	n	9
9	w	4
10	es<w>	13
11	old	10

Now, these 11 tokens will work as vocabulary

In GPT 2 or 3

around 57,000 tokens were
done using Binary Pair encoding

PAGE No.	
DATE	/ /

Stopping Condition / Criteria:

- ↳ can either be
 - i) token count or
 - ii) No. of iteration.

~~✱~~

token

↳ This is what GPT-2 used for tokenizing.

Here is code in VS code.

but this shows the exact TokenId used
for tokens used in training of gpt-2

∴ `<endof-text>` have tokenId 50256
which gpt loves to use.

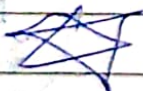
∴ Total words in english

↳ 170,000 ~ 200,000
words

Total tokens in Eng

↳ 50,256
tokens

(Not training data)



∴ we reduce

The size of data using BPE

if we do that

word based tokenizing, we require more space



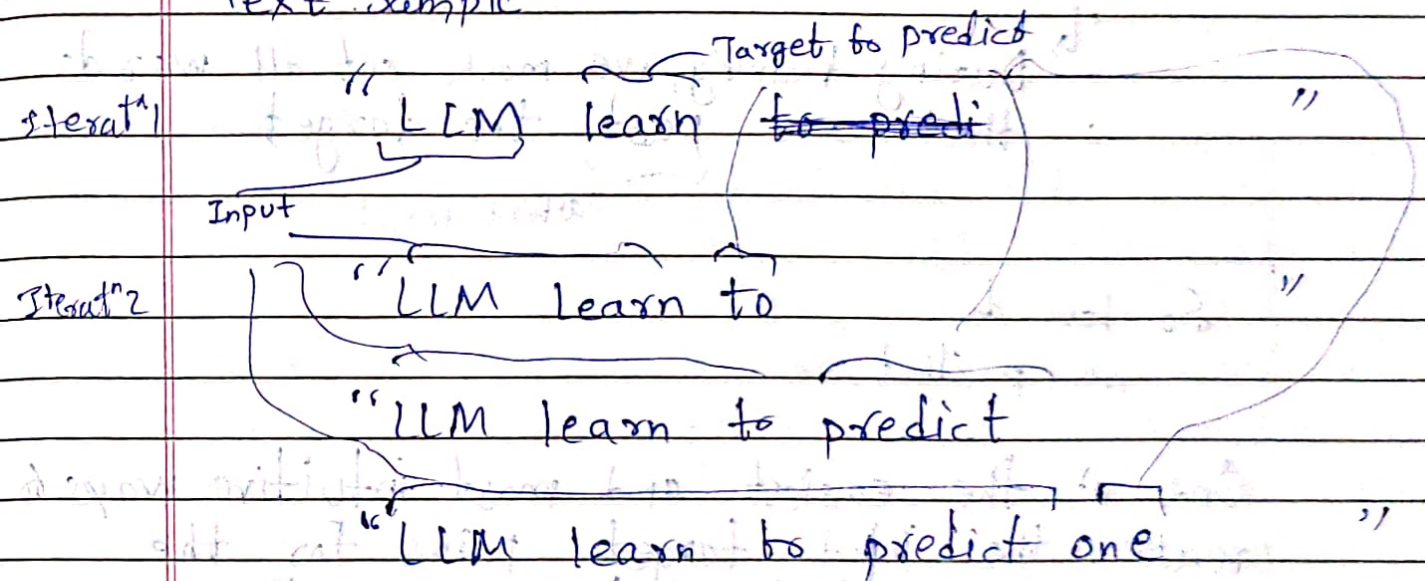
Also the problem of out of vocabulary
is also gone, in BPE.

	GPT 2	GPT 3	GPT 4/5
Vocab size	50,257 tokens	50,257 tokens	~100K tokens
Training data size	~8 billion tokens	~300 billion tokens	~estimated trillion of tokens

~~What~~ Input Target pairs.

What do these input-output pair looks like:

Text Sample



The LLM can't access words past the target output becomes input in next iteration

Self-supervised learning / auto-regressive / unsupervised learning

As we are not labeling, or doing anything, sentence structure itself is used to determine output, we just write code, which breakdown sentence itself breaks down sentence into input & output

Context size How many words, you want,
So that model can make its
output

PAGE No.	
DATE	/ /

So, in pretraining, we
always do supervised learning, cause the sentence
structure is exploited to create input-output pairs

So,

Given text sample,

↳ Extract input blocks as subsamples that serves
as input to the LLM.

↳ The LLM predict task during training is to
predict the next word that follows the input
block

↳ During training, we mask out all words
that are past the target.
↳ after the target

So how do
we predict,

One of the easiest and most intuitive ways to
create the input target pairs for the
next word prediction tasks is to
create

two variable x & y ,
where x contains the input token &
 y contains the targets,
which are input shifted by 1.

How many words, the model
should pay Attention, at
a time
to predict next word.

Context size =

PAGE NO.	
DATE	/ /

So,

if

input = 1 output = 2

input = 12 output = 3

input = 123 output = 4

input = 1234 output = 5

$X = [1, 2, 3, 4]$ input
 $Y = [2, 3, 4, 5]$ output

Ex.

$X = \text{enc_sample}[: \text{context_size}]$

$Y = \text{enc_sample}[\text{context_size}+1]$

`print(f "X: {x}")`

`print(f "Y: {y}")`

output:

$X = [290, 4920, 2241, 287]$

$Y = [4920, 2241, 287, 257]$

Processing the inputs along with the targets, which are the inputs shifted by one position, we can then create the next-word prediction tasks as follows.

for i in range(1, context_size+1):

context = enc_sample[:i]

desired = enc_sample[i]

`print(context, " - - -> ", desired)`

output:

$[290] - - -> 4920$

$[290, 4920] - - -> 2241$

$[290, 4920, 2241] - - -> 287$

$[290, 4920, 2241, 287] - - -> 257$

∴ In LLM, one input-output pair corresponds to the number of prediction tasks as set by context size.

We can convert those token id's into tokens using the tokenizer

So, input-target pairs are done, but we have to make it in very structured way

Now,

* Data loader

implementing data loader that iterates over the input dataset & returns the inputs & Targets as PyTorch tensors, which can be thought of as multidimensional array,

In particular,

we are interested in returning two 'Tensors':

- 1) An input tensor containing the text that the LLM sees &
- 2) Target tensor that includes the targets for the LLM to predict.

Extra Know:
PyTorch's data primitive.
Dataset stores sample & corresponding labels

Dataloader: wraps an iterable around the Dataset to enable easy access to the sample

* Datasets & Dataloaders

↳ Enable you to load & process data in much more efficient & compact manner

Implementing Data loader.

↳ we ~~are~~ will implement a data loader that fetches input-output target pairs using a sliding window approach.

Sample text

Apple	Banana	chicko	orange	Dog	cat	rat
Goat						

Context window of 4

Tensor containing the input

```
X = tensor([["Apple", "Banana", "chicko", "orange"],  
            ["Dog", "cat", "rat", "goat"],  
            ["", "", "", ""],  
            ["", "", "", ""]])
```

Tensor containing the output (which is shifted by one)

```
Y = tensor([["Banana", "chicko", "orange", "Dog"],  
            ["cat", "rat", "goat", ""],  
            ["", "", "", ""],  
            ["", "", "", ""]])
```


To implement efficient dataloaders, we collect inputs in a tensor X , where each row represents one input context.

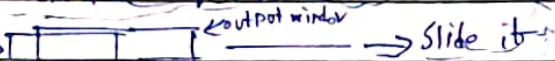
The Second tensor Y contains the corresponding prediction targets (next word), which are created by shifting the input by one position.

$\therefore$ 50th row of X input $\rightarrow$ 30th row Y output
& it will be
50th input-output pairs

& in each input-output pair there are four prediction tasks

$\therefore$ For efficient dataloader implementation, we will use Pytorch's built in Dataset & Dataloader classes.

Step 1: Tokenizing entire text

Input window  Slide it

Step 2: Use a sliding window to chunk the book into overlapping sequences of max length

Step 3: Return the total number of rows in the dataset.

Step 4: Return a single row from the dataset.

also, by this,

Dataset is done,

but,

For using data loader,

Dataset → types { Map-style dataset (we are using)
 Iterable-style dataset

∴ In map style dataset, we use,
 the `getitem()` & `len()`
 to use `Dataloader`.

→ It will show what kind of
 input & output do we have

- ∴ We use Pytorch Dataset class.
- ∴ Each row consists of number of token IDs (based on max length) assigned to an input chunk tensor.
- ∴ The target chunk tensor contains the corresponding targets.

Code in VS Code.

The following code will use the `GPTDatasetV1` to load the inputs in batches via a Pytorch `Dataloader`:

Step 1: Feeding to Dataloaders!

step 1: Initialize the tokenizer.

Step 2: Create dataset

Step 3: drop_last = True, drops the last batch if it is shorter than the specified batch size to prevent loss spikes during training

Step 4: The number of CPU processor to use for processing.

Code in VS code

Why we use Data loader

↳ help in parallel processing
↳ & also analyse multiple batches at a time.

• batch size *: The no. of batches model processes once before updating its parameters.

So to make sure model update its parameters quickly

The data is usually chunk into batches

if batch size = 4 (Default)

↳ After analysing 4 batches, the model will update its parameter rather going to entire dataset.

In deep learning know:

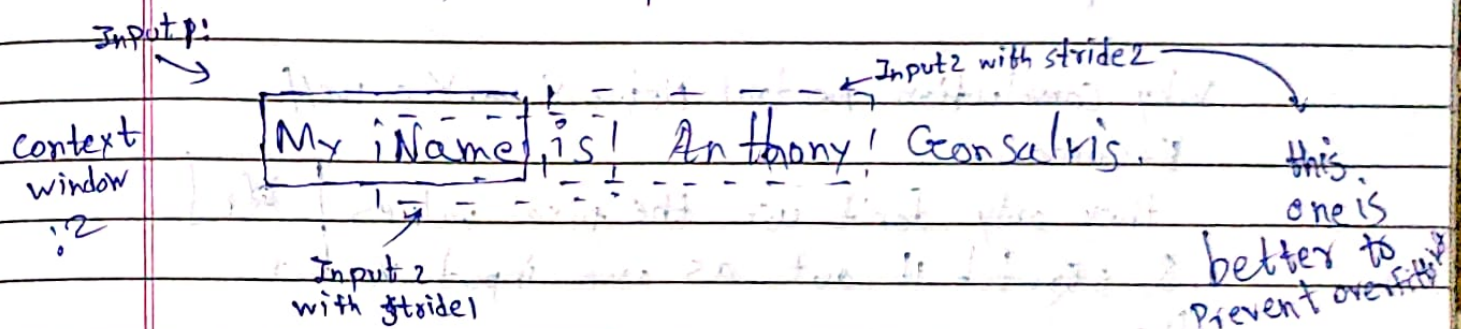
Small batch size require less memory during training
but lead to more noisy model updates
The batch size is a trade-off

PAGE No.	
DATE	/ /

num_workers : basically for parallel processing,
on different threads of your CPU.

$\&$ Dataloader allows, these,
batch size, drop_last, shuffle, num_workers
to use, without it, it is hard to implement

Stride: ~~a slider~~ helps to slide the input window
by n places for next batch



The stride setting dictate the no. of positions
the inputs shifts across batches, emulating
a sliding window approach.

$\&$

Stride No. = Context window No

$\hookrightarrow$ Most efficient (used in training)
less memory

Stride No < Context window No

$\hookrightarrow$ More optimal for accuracy, keep overlapping
chunk so info isn't lost at boundary.
Context preserved

(used in ~~int~~ embedding/retrieval)

check the code in VS

* Vector / Token / word Embedding

computer can't understand word,

so

we require it to represent numerically

However, Numbers can't associate the semantic meaning

Eg, cat $\rightarrow$ 13 Kitten $\rightarrow$ -40

Why Convolution Neural Network work so well

Because

they only don't use the pixel values

& stretch it out as one input vector

they

actually encode the spatial relationship between the pixels,

eg. There are two eyes, close to each other

That is an info which is contained in the image itself
& we should exploit this info when we give / feed
input to the model

That's why, Convolutⁿ neural network exploit the
spatial relation between pixel & exploit this
information inherently present in an image

So in text format,
words have meaning, sentences carry meaning,
& there are some words which are closer in meaning
to other words, this is the inherent advantage in the
text which we need to exploit

We need to exploit the fact that cat &
kitten are closer to each other.

& This is the optimal way

• One hot encoding

"dog" $\rightarrow$ [00000010010]

"Puppy" $\rightarrow$ [000000001000]

One hot encoding Also fails to capture
semantic relationship.

'Words have meaning, & why do we exploit
this meaning when we construct the inputs
to be given to LLM'.

So assigning ~~random~~ token of one-hot-coding does not work

Semantically Similar words should have similar vector

B-D
vector

	"dog"	"cat"	"apple"	"banana"
has a tail	23	31	1	2
is-eatable	2	3	22	38
has 4-legs	19	21	0	0
make-sound	12	18	0.5	0.2
is a pet	35	31	5	7

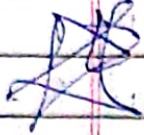
more related

more related

If we represent,

Word $\rightarrow$ As a vector []

&

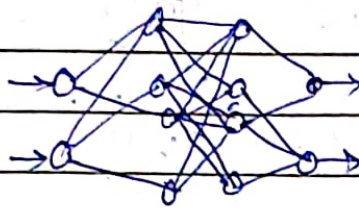


If we construct vector in good manner,
vector can capture semantic meaning

we

have to train Neural Network to
Create Vector Embedding

"dog"
"cat"
"Apple"
"banana"



[23, 2, 19, 12, 35]
[31, 3, 25, 18, 31]
[1, 220, 0.5, 5]
[2, 38, 0.2, 7]

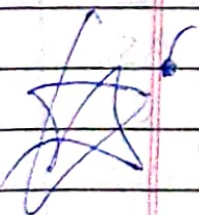
Check out ~~you~~ that

Google collab, of
that ~~vec~~

$\rightarrow$ word to vec 300

Already
created
vector embedding
dataset

in
google collab



By measuring Magnitude difference
between the two vectors, we can
determine how related they are!

How are Embedding created For LLM?

a) Initialize embedding weights -
First basics,

Vocab

□ → 1

□ → 2

□ → 3

□ → 4

→ Token ID

converted

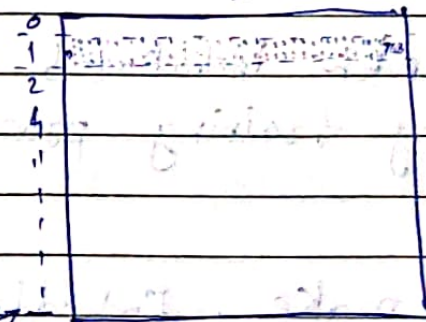
into embedding
vectors

Embedding
matrix

Token ID

Vector dimension = 768

For GPT 2



For GPT-2 → 50,257 → vocab-size.

For N Tokens

∴ for Token Id 1 → 768 NumDim

50000 →

∴ Weigh of this Embedding Matrix
is

$$\text{Weight} = 50257 * 768$$

&

This is called as Embedding layer weight Matrix.

So Token id → Embedding layer weight Matrix

Step I: Initialize embedding weights with random value.

So, all these 50257×768 got random value

Step II: The initialization serves as the starting point for the LLM learning process.

Step III: The embedding weights are optimised as part of the LLM training process

concept of Back propagation of Neural Network is implemented to optimize all of these weights of Embedding layer & weight matrix

Those random initialized values, gets optimised during training process.

So
lets make, Embedded layer Matrix for 6 tokens having 3 dimensions

Vocab-size = 6

output_dim = 3

torch.manual_seed(123)

embedding_layer = torch.nn.Embedding(Vocab-size, output_dim)

print(embedding_layer.weight)

Simple lookup table that stores Embedding of words fixed dictionary words & size(dimensions)

This is 3D vector with 12 tokens

Output: Parameter containing:

tensor([[[2.134, -1.341, 1.234],
[1.234, 1.041, 1.45],
[..., ..., ...]]])

Each row represent 1 token

6×3

all initialized with random value later updated

Here the Weight matrix of embedding layer contains small, random values.

These values are optimised during LLM training as a part of the LLM optimization itself. as we will see in upcoming chapters.

We can see that the weight matrix has 6 rows & 3 columns

There are 6 rows for each six possible tokens in vocabulary.

And there is 1 column for each of 3 embedding dimensions.

we can access the vector of particular token id

```
print (embedding_layer (torch.tensor([3])))
```

output : tensor([[-0.4015, 0.9666, -1.1481]], grad_fn=)

We can also use group of token ids.

```
input_ids = torch.tensor([2, 3, 5, 1])
```

```
print (embedding_layer (input_ids))
```

output

```
tensor([
  [ ..., ..., ...],
  [ ..., ..., ...],
  [ ..., ..., ...],
  [ ..., ..., ...]])
```

token 2
token 5

∴ The embedding layer is look up operatⁿ that retrieves rows from the embedding layer weight matrix using a token ID.

So,

What an Embedding layer actually do

* Suppose we have ^{following} training example.

$idx = \text{torch.tensor}([2, 3, 1])$

* Embedding dimension = 5

* $\text{embedding} = \text{torch.nn.Embedding}(4, 5)$

* $\text{print}(\text{embedding.weight})$

Parameters containing

$\text{tensor}([[u, v, w, x, y],$

$[u, v, w, x, y],$

$[u, v, w, x, y],$

$[u, v, w, x, y],$

$[u, v, w, x, y]$

$\text{required_grad=True})$

* $\text{embedding}(idx)$

$\text{tensor}([[u, v, w, x, y],$

$[u, v, w, x, y],$

$[u, v, w, x, y]$

$\text{grad_fn} = \langle \text{EmbeddingBackward0} \rangle$

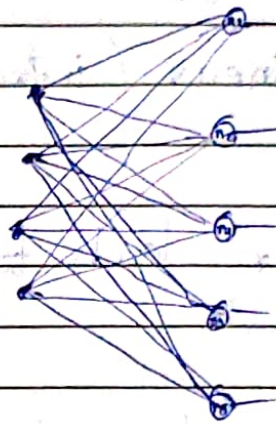
This is same as Neural Network linear layer

Ex.

exo one-hot-encoding

token 1 $\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \end{bmatrix}$
 token 2 $\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \end{bmatrix}$
 token 3 $\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix}$

X



5 Neurons (Change 5 Dimension)

output = $X \cdot W^T$

$X \in$ input matrix
 $W^T \in$ weight matrix

$$W^T = \begin{bmatrix} W_{11} & W_{21} & W_{31} & W_{41} & W_{51} \\ W_{12} & W_{22} & W_{32} & W_{42} & W_{52} \\ W_{13} & W_{23} & W_{33} & W_{43} & W_{53} \\ W_{14} & W_{24} & W_{34} & W_{44} & W_{54} \end{bmatrix}$$

weight of Neuron n_1
 for input
 i.e. 0, 0, 1, 0, 0

weight of
 n_5
 for input 0
 i.e. token 2

every Neuron have four
 weight associated with it

$X \cdot W^T \Rightarrow$ you will get Matrix of 3 rows X 5 column

Exactly same as what we got when we used
 embedding layer, i.e.

tensor ($[L, n, n, n, n]$,
 $[L, n, n, n, n]$,
 $[L, n, n, n, n]$) $\Rightarrow$ $\text{grad_fn} = \langle \text{MmBackward0} \rangle$

So we can use,

Embedding layer $\rightarrow$ `torch.nn.Embedding (vocabsize, dimension)`

or

Linear layer $\rightarrow$ `torch.nn.Linear (in-feature, out-feature, bias=True, device=None)`

* Both layer leads to same output but

* Embedding layer is much more computationally efficient, since NN linear layer has many unnecessary multiplications with zero.

$\therefore$ GPT-2 has an embedding weight matrix of

50257×768

but the

model isn't just that - most parameters comes from Attention + MLP layer.

1) Until Now, we looked at token embedding.

2) In the embedding layer, the same token ID gets mapped to the same vector representation, regardless of where the token ID is positioned in the input sequence.

Eg. Cat sat on mat

~~The~~ on the mat the cat sat

Cat $\rightarrow$ Same token ID

have same vector

The, same token ID's result in the same embedding vectors.

3) It is helpful to inject additional position information to the LLM.

4) So, How we give positional info, there are two ways.

Positional Embedding

Absolute

Relative

For each position in input sequence, a unique embedding is added to the token's embedding to convey it's exact location

The emphasis is on the relative position or distance between tokens, The model learns the relationship in terms of 'how far apart' rather than at which exact position

ex. The cat sat on mat, on mat cat sat

Advantage: model can generalize better to sequence of varying length, even if it

[1, 2, 1, 1, 1, 4]

Absolute Embedding

token: the cat sat on a

Token Embed: $\begin{bmatrix} 1, 2, 3 \end{bmatrix}$ $\begin{bmatrix} 4, 5, 6 \end{bmatrix}$ $\begin{bmatrix} 7, 8, 9 \end{bmatrix}$ $\begin{bmatrix} 0, 1, 4.5 \end{bmatrix}$

+

Position Embed: $\begin{bmatrix} 1.1, 1.2, 1.3 \end{bmatrix}$ $\begin{bmatrix} 1.2, 1.1, 1.4 \end{bmatrix}$ $\begin{bmatrix} 1.1, 1.1, 1.0 \end{bmatrix}$ $\begin{bmatrix} 1.0, 1.0, 1.5 \end{bmatrix}$

= Input Embed: $\begin{bmatrix} 2.1, 2.4, 4.3 \end{bmatrix}$ $\begin{bmatrix} 5.2, 6.1, 7.4 \end{bmatrix}$ $\begin{bmatrix} 8.1, 9.1, 10 \end{bmatrix}$ $\begin{bmatrix} 1, 2, 3 \end{bmatrix}$

Position vectors have same dimension as the original token Embedding.

Comparatively Relative embedding is more better

• Both types of positional ~~embedding~~^{embedding/encoding} enables LLM's to understand the order & relationship between tokens, ensuring more accurate & context aware predictions.

• The choice between two depends on specific application & nature of data being processed.

Absolute: Suitable when Fixed order of tokens is crucial, such as sequence generation

Attention BAU
Read the paper

used formula:

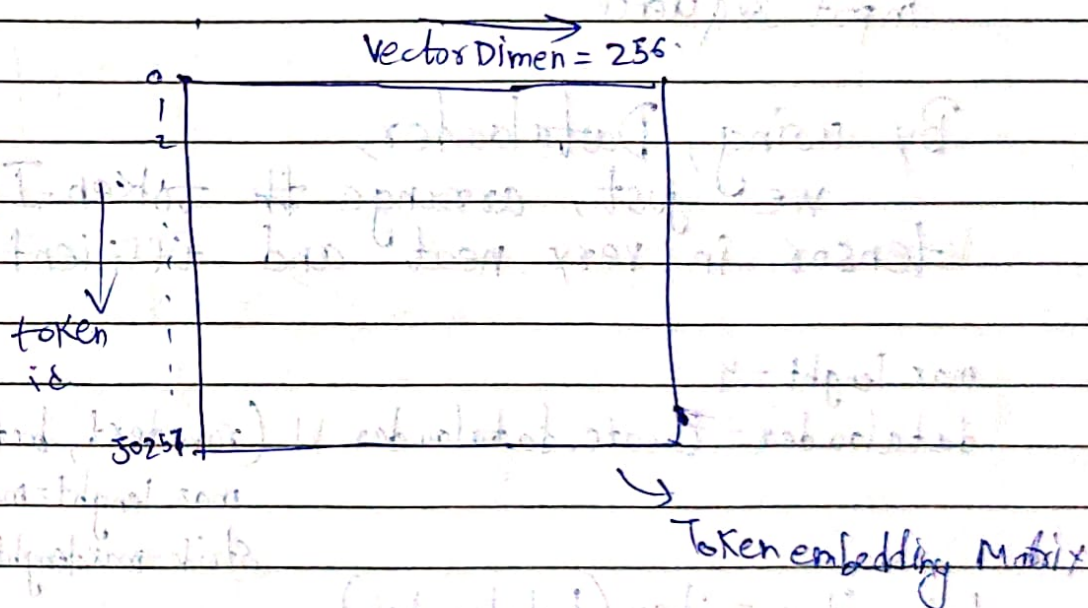
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Positional Embed.

Relative: Suitable for tasks like language modeling over long sequences, where the same phrase can appear in different parts of the sequence.

- OpenAI's GPT model use absolute positional embedding that are optimized during the training process. This optimization is part of the model training itself.

- Implementing positional vector



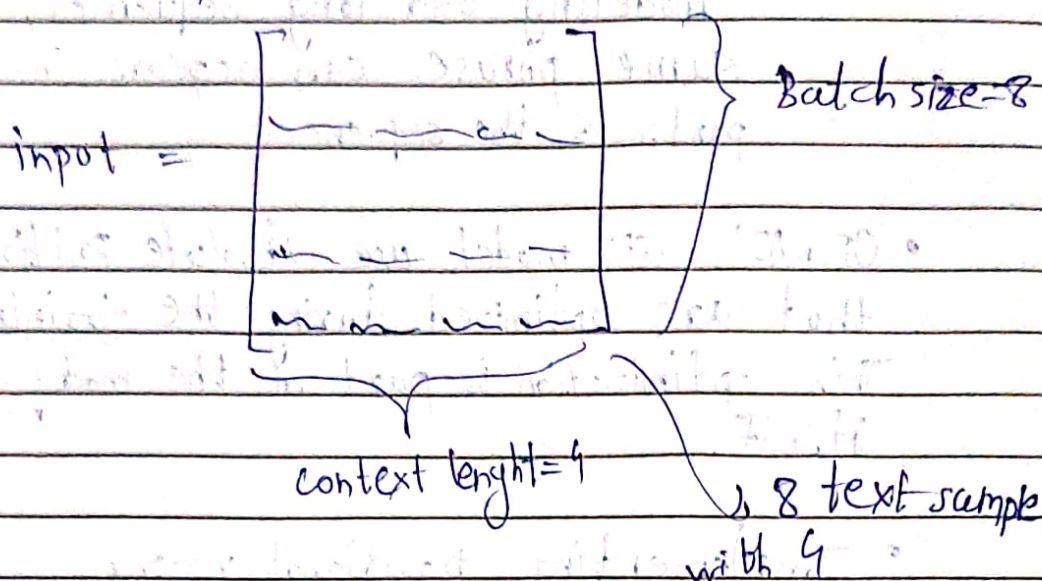
Vocab size = 50257

output dim = 256

token embed ding layer = torch.nn.Embedding(vocab size, output dim)

Now using dataloader, by which we embed each token in each batch into a 256-dimensional vector. if we have a batch size of 8 with 9 token each, the result will be an $8 \times 9 \times 256$ tensor.

• Create input data from dataloader



Each row represent, 1
input sequence

By using, Dataloader,
we just, arrange the token-ID's in
tensor in very neat and efficient manner

max-length=4

dataloader = Create dataloader-V1 (raw-text, batch-size=8,
max-length=max-length, ~~stride~~
stride=max-length, shuffle=false)

data-iter = iter(dataloader)

input, targets = next(data-iter)

print("Token ID's: \n", inputs)

print("\n. input shape: \n", input.shape)

Output:

Token IDs:

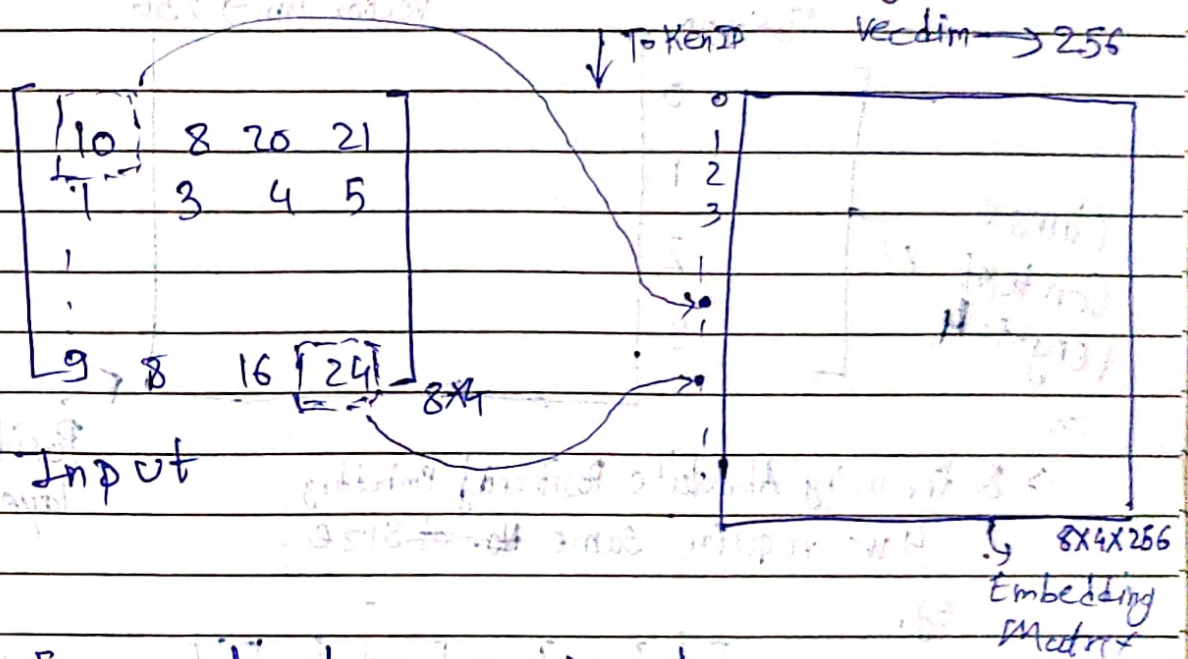
tensor ([[40, 367, 2885, 1464],
[1807, 3619, 402, 271],
[
[284, 3285, 326, 11]])

8-batch
size.

input₂.shape:
torchsize ([8,4])

As we see,
the token ID tensor is 8x4-dimensional,
meaning that the
data batch consists of 8 text samples
with
4 token each.

So we have input tensor, by using Dataloader



$\therefore$ For each token in input,
One embedding vector of 256 length is generated

check: Now

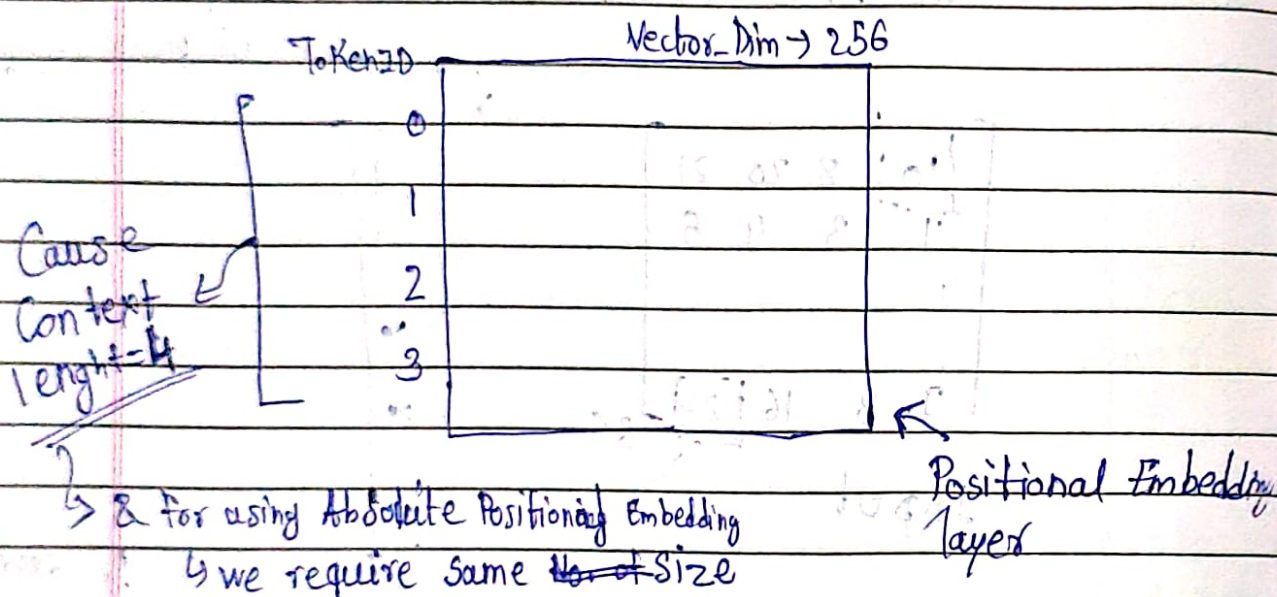
using,
~~token emb~~ Embedding layer to embed these
token IDs into 256-dimension vectors.

token_embeddings = token_embedding_layer(inputs)
print(token_embeddings.shape)

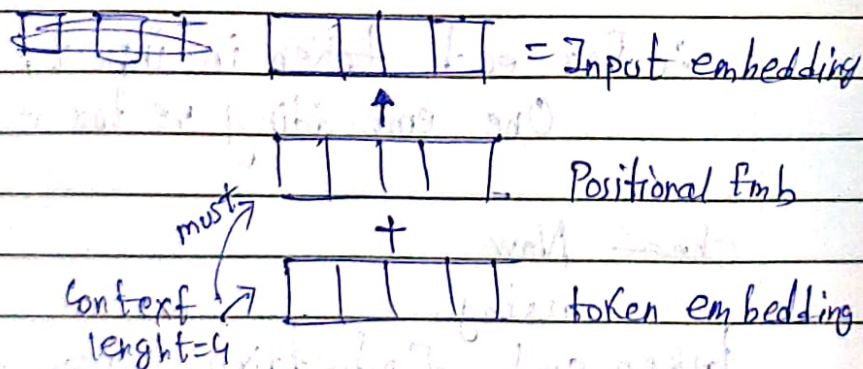
output : torch.Size([8, 4, 256])

For using Positional Embedding/encoding

① Create another embedding layer for Positional encoding



eg.



For token embedding,

token_embedding_layer = torch.nn.Embedding(max_length, dim)

For position Embedding

Positional_embedding_layer = torch.nn.Embedding(context_len, dim)

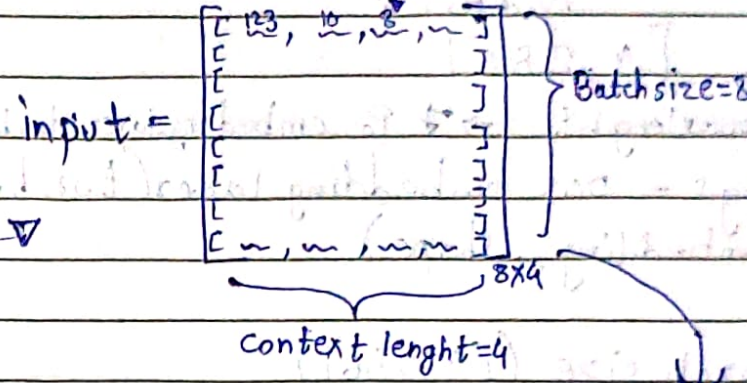
cat=123

input
↓
split (pre)
↓
token
↓
vocab
↓
dataset
↓
data loader

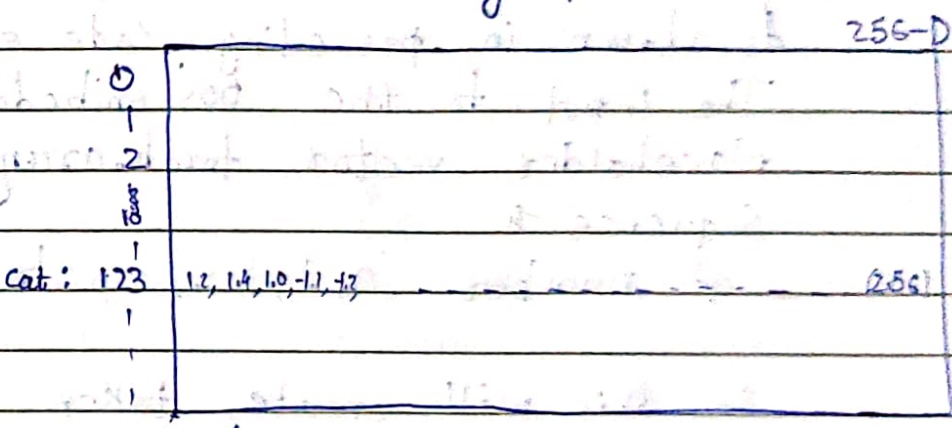
(12)



token-Id



Embedding layer



we just need
to encode
diff. pos. in

Suppose, First batch is

[123, 10, 8, 20]

∴ We need to add one
position vector to each
of these 4 tokens

(Why we not use different Position embed,
for each token
Cause for each input, we just have
to check, wheater it is in first,
Second, third or fourth place

∴ The same positional
Embeddings are applied to
each input of 4 tokens.
(because there are only 4 position

Hence we need
4 positional
embedding Vector

∴ We have to create position Embed
once.
& then for every batch, the
Same posital embedding is used

⑬ Generate the 4 positional Embedding vector & from the positional Embedding Matrix

[4, 256]

Context length = max length. $\rightarrow$ Pos-embedding-layer = torch.nn.Embedding(Context

Pos embeddings = pos-embedding-layer(torch.arange(max length))
print(pos_embeddings.shape)

output: torch.size([4, 256])

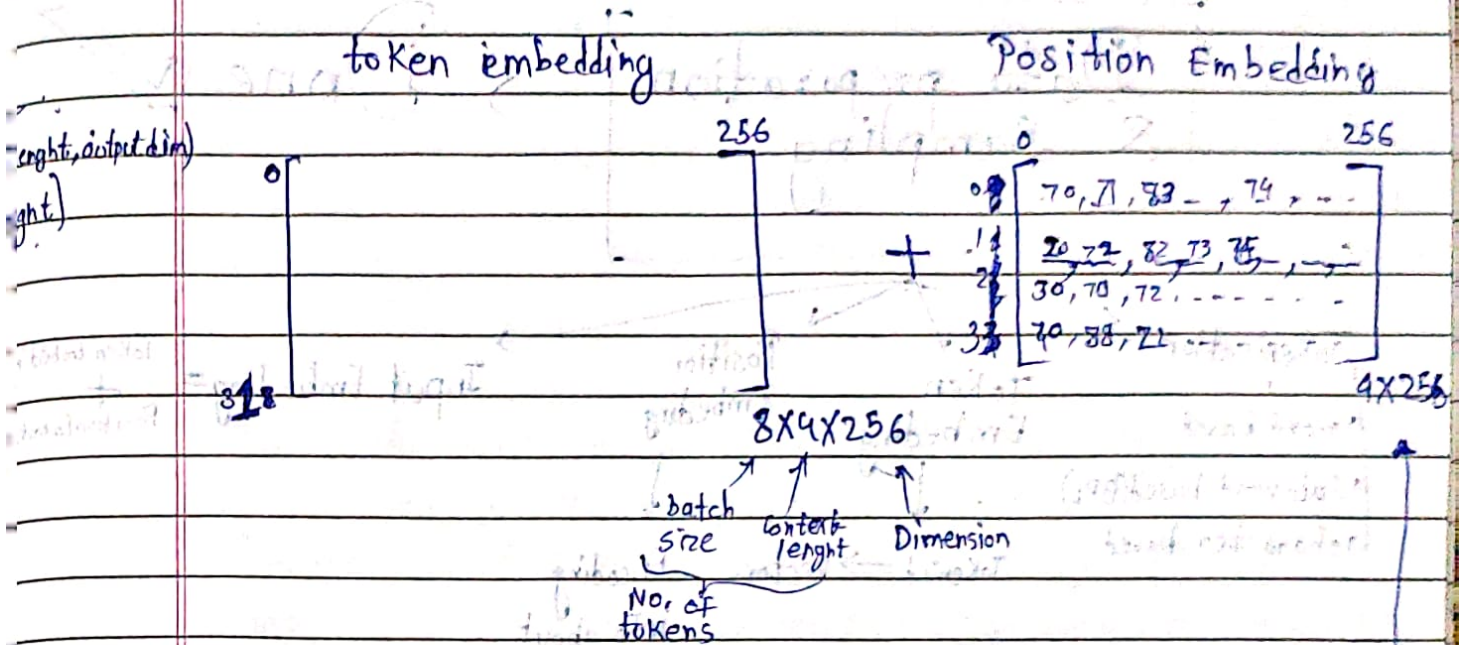
As shown in preceding code example,
The input to the pos-embeddings is usually a placeholder vector torch.arange(Context length)
Sequence of numbers 0, 1, ..., up to maximum length - 1

So this will create, token id, 0, 1, 2, 3
& look up

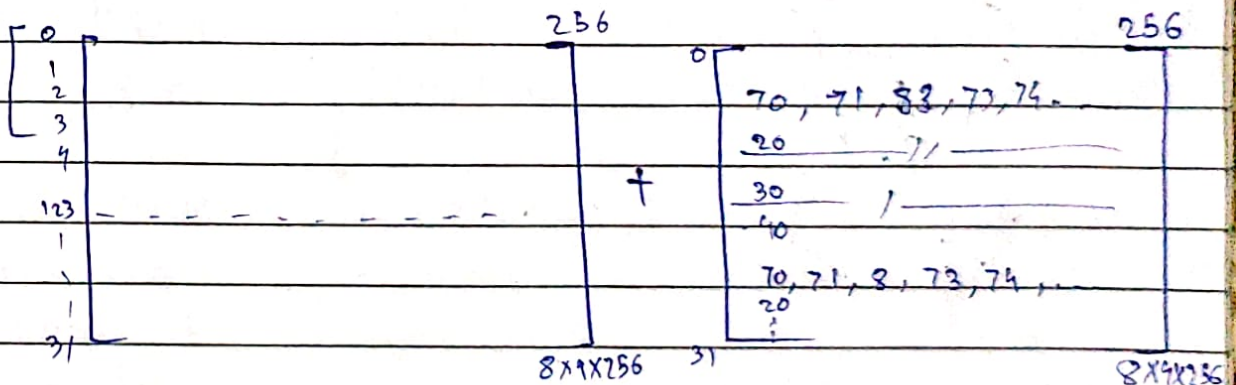
the positional embedding table, & generate these 4 embedding vectors.

As we can see, the positional embedding tensor consists of four 256-dimensional vectors.

⑨ Add the token embedding + Position embedding



Broadcasting operation done by python
It duplicate the values, in smaller one downward.



0 [] + 0 [70, 71, 83, ...]
1 [] + [20, ...]
2 [] + [30, ...]
3 [] + [40, ...]

For GPT2

(50, 256 x 1024 x 768)

= Input Embeddings (8 x 4 x 256)

Go To This

20

Data preparation & Sampling

Done ✓

Tokenization

- word based
- Sub-word based (BPE)
- character based

Token Embedding

TokenId → Vectors

Position Embedding

Encoding info about Position

Input Embedding =

$$\text{Token Embedding} + \text{Position Embedding}$$

